

DP-Based Partitioning for Minimizing Maximum Load

1 Background and Objective

In many scheduling and load-balancing problems, especially in the context of partitioning deep neural network computations across multiple devices, the goal is to minimize the **maximum load metric**. In our framework, the load metric is defined as the execution time for a block of layers (or tasks). Minimizing the maximum load means that we want to partition the work such that the *slowest* (or most costly) segment has as low an execution time as possible. This is critical because the overall performance is often determined by the bottleneck—the segment with the maximum load.

2 Mathematical Formulation and Recurrence Equation

Let the model be divided into L layers that must be executed sequentially. We wish to partition these layers into K contiguous segments such that the maximum execution time across all segments is minimized. Define:

- $cost[i][j]$: The cost (execution time) of processing layers i through j on the best (least costly) available device.
- $DP[i][k]$: The minimum possible maximum cost when partitioning layers 0 through i into k segments.

For the base case, when $k = 1$, we have:

$$DP[i][1] = cost[0][i] \quad \text{for } i = 0, 1, \dots, L - 1.$$

For $k > 1$, the recurrence is:

$$DP[i][k] = \min_{0 \leq x < i} \left\{ \max \left(DP[x][k - 1], cost[x + 1][i] \right) \right\}.$$

This recurrence means that we consider every possible split point x for the layers 0 to i . The first segment is taken as the optimal partitioning of layers 0 to x into $k - 1$ segments, and the second segment is the block from $x + 1$ to i . We then choose the split that minimizes the maximum cost between these two parts.

3 Example: $L = 5$ Layers, $K = 2$ Segments

We now illustrate the algorithm with a hypothetical cost matrix.

3.1 Precomputed Cost Matrix

Assume we have the following costs for executing contiguous blocks on the best node:

For instance, $cost[0][2] = 8$ means that processing layers 0, 1, and 2 together takes 8 time units.

$i \setminus j$	0	1	2	3	4
0	2	5	8	12	15
1	–	3	6	10	13
2	–	–	4	7	9
3	–	–	–	5	8
4	–	–	–	–	6

Table 1: Cost Matrix: $cost[i][j]$

i	$DP[i][1]$
0	2
1	5
2	8
3	12
4	15

Table 2: DP Table for $k = 1$

3.2 DP Table Initialization ($k = 1$)

For a single segment, the cost is:

$$DP[i][1] = cost[0][i].$$

Thus, we obtain:

3.3 Filling the DP Table for $K = 2$

We now partition the 5 layers into 2 segments. For each ending index i (from 1 to 4) we try every possible split point x ($0 \leq x < i$) and compute the candidate:

$$\text{Candidate} = \max \left(DP[x][1], cost[x+1][i] \right).$$

For $i = 1$:

- $x = 0$: $DP[0][1] = 2, cost[1][1] = 3 \Rightarrow \text{Candidate} = \max(2, 3) = 3$.

Set:

$$DP[1][2] = 3, \quad split[1][2] = 0.$$

For $i = 2$:

- $x = 0$: $DP[0][1] = 2, cost[1][2] = 6 \Rightarrow \text{Candidate} = \max(2, 6) = 6$.
- $x = 1$: $DP[1][1] = 5, cost[2][2] = 4 \Rightarrow \text{Candidate} = \max(5, 4) = 5$.

Set:

$$DP[2][2] = 5, \quad split[2][2] = 1.$$

For $i = 3$:

- $x = 0$: $DP[0][1] = 2$, $cost[1][3] = 10 \Rightarrow \text{Candidate} = \max(2, 10) = 10$.
- $x = 1$: $DP[1][1] = 5$, $cost[2][3] = 7 \Rightarrow \text{Candidate} = \max(5, 7) = 7$.
- $x = 2$: $DP[2][1] = 8$, $cost[3][3] = 5 \Rightarrow \text{Candidate} = \max(8, 5) = 8$.

Set:

$$DP[3][2] = 7, \quad split[3][2] = 1.$$

For $i = 4$:

- $x = 0$: $DP[0][1] = 2$, $cost[1][4] = 13 \Rightarrow \text{Candidate} = \max(2, 13) = 13$.
- $x = 1$: $DP[1][1] = 5$, $cost[2][4] = 9 \Rightarrow \text{Candidate} = \max(5, 9) = 9$.
- $x = 2$: $DP[2][1] = 8$, $cost[3][4] = 8 \Rightarrow \text{Candidate} = \max(8, 8) = 8$.
- $x = 3$: $DP[3][1] = 12$, $cost[4][4] = 6 \Rightarrow \text{Candidate} = \max(12, 6) = 12$.

Set:

$$DP[4][2] = 8, \quad split[4][2] = 2.$$

The resulting tables for $k = 2$ are:

i	$DP[i][2]$
1	3
2	5
3	7
4	8

Table 3: DP Table for $K = 2$

i	$split[i][2]$
1	0
2	1
3	1
4	2

Table 4: Split Table for $K = 2$

The split table records, for each subproblem, the optimal index x where the partition should occur. For example, $split[4][2] = 2$ indicates that for layers 0 to 4, the optimal split is after layer 2.

3.4 Backtracking to Recover the Partition

To retrieve the final segmentation, we backtrack from $DP[4][2]$:

1. Start at $i = 4$ with $k = 2$: We have $split[4][2] = 2$. This means the optimal split is after layer 2. Thus, the second segment covers layers 3 to 4.
2. The first segment then covers layers 0 to 2.

The final partition is:

- **Segment 1:** Layers 0, 1, and 2.
- **Segment 2:** Layers 3 and 4.

The overall makespan (i.e., the maximum segment cost) is:

$$\max(cost[0][2], cost[3][4]) = \max(8, 8) = 8.$$

4 Pseudocode Summary

```
function partition_layers(Layers, K):
    L = length(Layers)
    // Precompute cost matrix:
    for i = 0 to L-1:
        for j = i to L-1:
            cost[i][j] = compute_cost(Layers[i..j])

    // Initialize DP table for 1 segment:
    for i = 0 to L-1:
        DP[i][1] = cost[0][i]

    // Fill DP table for k = 2 to K:
    for k = 2 to K:
        for i = 0 to L-1:
            for x = 0 to i-1:
                candidate = max(DP[x][k-1], cost[x+1][i])
                if candidate < DP[i][k]:
                    DP[i][k] = candidate
                    split[i][k] = x

    // Backtrack to recover partitions:
    partitions = []
    current_k = K
    current_i = L - 1
    while current_k > 1:
        x = split[current_i][current_k]
        partitions.prepend((x+1, current_i))
        current_i = x
        current_k -= 1
    partitions.prepend((0, current_i))

    return partitions, DP[L-1][K]
```

5 Incorporating a Transfer Penalty

If data transfers incur an overhead, you can modify the candidate calculation as:

$$\text{Candidate} = \max(DP[x][k-1], cost[x+1][i] + \text{transfer_penalty}).$$

This extra term accounts for the additional cost when a new segment is started (i.e., data needs to be moved between nodes). This adjustment becomes more significant on high-end systems where computation is fast but data transfer times are non-negligible.

6 Conclusion

We have presented a DP-based partitioning algorithm aimed at minimizing the maximum load (execution time) across segments. The key steps are:

- **Cost Matrix:** Precompute the cost for every contiguous block.
- **DP Initialization:** Set $DP[i][1] = cost[0][i]$.
- **DP Recurrence:** For $k \geq 2$, use

$$DP[i][k] = \min_{0 \leq x < i} \{ \max(DP[x][k-1], cost[x+1][i]) \}.$$

- **Split Table:** Record the optimal split points.
- **Backtracking:** Recover the optimal segmentation.
- **Transfer Penalty (Optional):** Adjust the recurrence to account for data movement overhead.

This approach ensures that the maximum execution time (the bottleneck) is minimized, thereby balancing the load effectively across available computing resources.